《FPGA 工具与基础设计》

《FPGA 工具与基础设计》 1
一：仿真工具（Modelsim） 3
1.1 仿真库 3
1.2 仿真分类 4
1.2.1 独立仿真 5
1.2.2 自动化仿真 9
1.2.3 联合仿真 10
二：HDL 设计基础 12
2.1 HDL 建模 13
2.1 数据流建模 14
2.2 行为建模 15
2.2.1 组合电路建模 16
2.2.2 时序电路建模 17
2.2.3 关于组合电路和时序时序特征 19
2.3 结构化建模 20
2.3.1 结构化建模设计案例 21
三：基础电路设计 23
3.1 触发器 23
3.2 多路器 25
3.3 译码器 26
3.4 计数器 28
3.5 分频器 29
3.5.1 时钟信号与时序电路相关联的属性 29
3.5.2 分频器电路 30
3.5.2.1 偶分频器（K=2,4,6,8,10.....） 31
3.5.2.2 奇分频器（K=1,3,5,7,9,11.....） 32
3.5.3 时钟分类 34
3.6 加法器 37
四：QUARTUSS II 集成开发环境 39
4.1 集成开发环境基本概念与分区 39
4.2QUARTUSS II 软件版本与 FPGA 开发工程属性 42
4.3 蜂鸣器驱动 42

一：仿真工具（Modelsim）

仿真是 HDL 设计的检查电路设计是否符合我们预期的一种手段，非常高效且使用，其属于验证学的一个分支，属于动态验证的。仿真需要结合仿真器，通过激励文件的激励信号值，观测响应信号的变化。

仿真的原理就是通过对电路系统的激励信号作用，观测响应信号的变化是否正确，通过虚拟仿真器进行动态模拟，数字电路设计中常用的仿真器有：

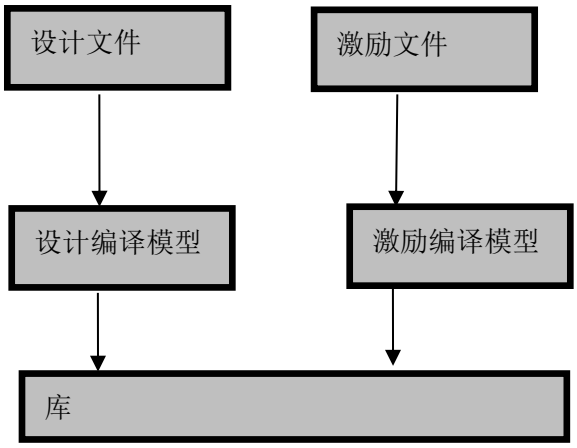
EDA 仿真器	应用目标	操作系统支持	HDL 语言支持
Modelsim	FPGA/CPLD(半定制开发)	windows	verilog/vhdl
Questasim	验证	windows/linux	system verilog system C
vcs	ASIC（全定制开发）	linux	system verilog system C verilog/vhdl

在 FPGA 开发中主要以 Modelsim 作为仿真器的标杆，对应不同的场景有不同的版本，根据自己情况下载适合自己的版本。

免费版：完全免费，比如：modelsim-altera
教育版：完全免费
专业版：收费，可破解
企业版：收费

1.1 仿真库

任何仿真都需要将设计输入文件经过编译，生成编译模型（仿真编译文件），这些编译模型需要存放在库中，所谓的库就是一个存储空间虚拟映射，这段存储空间的物理映射名叫做库。



仿真库原理示意图

Note:任何设计输入文件都必须经过编译为仿真模型才能进行仿真

1.2 仿真分类

在数字电路设计中，通常仿真分为 3 钟，每个仿真都有自己的优缺点，实际中我们根据需要的情况选择适合自己的仿真方式。

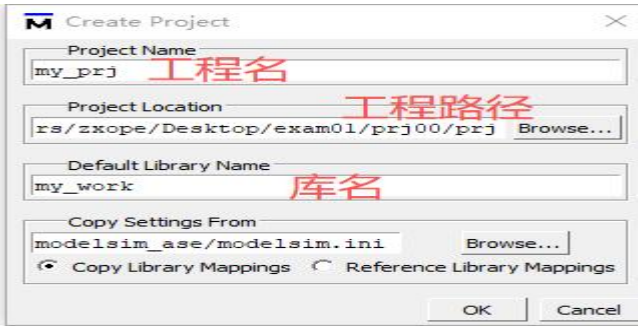
仿真方式	描述	特点
独立仿真	不依赖于任何其他工具，只需要仿真	优点： 不需要其他软件参与，节省仿真时间 缺点： 有的必要的仿真模型需要手动编译

	器即可完成仿真。	
联合仿真	仿真器 + 其他工具	优点:通常通过集成开发环境直接调用仿真器，所有的仿真库比编译在后天自动完成，对新手友好 缺点：需要同时掌握多个工具
自动化仿真	通过脚本命令完成自动化仿真	通过手动编写脚本完成自动仿真的一种仿真形式，比如：modelsim 的脚本为 do 脚本

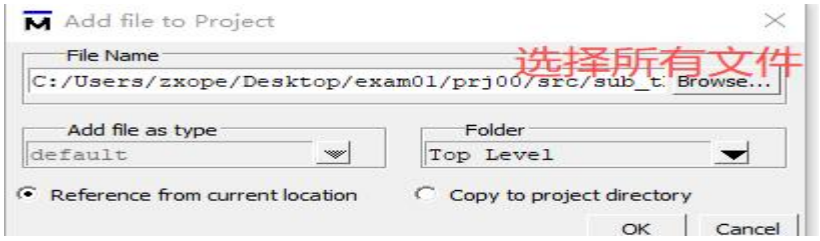
1.2.1 独立仿真

第 01 步：完成设计文件和激励文件的编写

第 02 步: 新建仿真工程



第 03 步: 添加所有参与仿真的文件

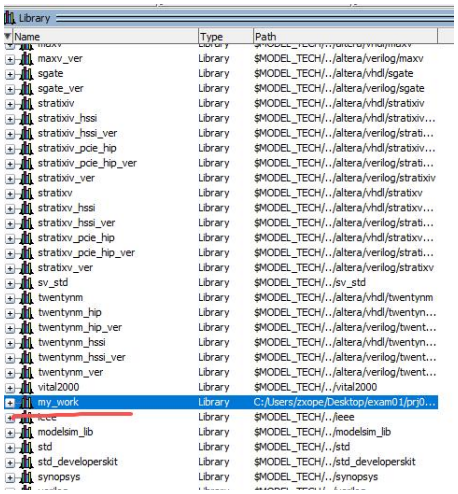


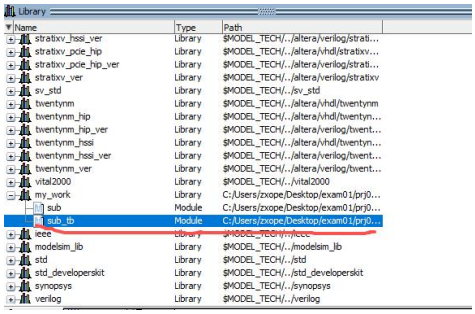
第 04 步: 编译所有文件

编译所有文件，如果有语法错误，则修改代码，没有的话编译后的对应的所有的编译模型自动保存在当前工程所对应的库中，比如：my_work

第 05 步: 启动仿真

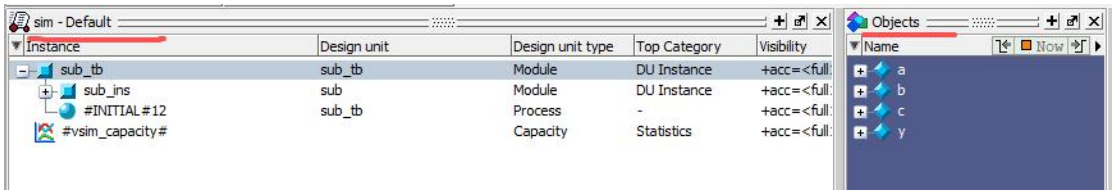
找到当前工程对应的库，比如 my_work，找到 tesetbench 的编译模型，启动仿真。





第 06 步: 添加需要观测信号到波形显示窗口

找到仿真模型管理窗口（比如：modelsim-sim）选中需要添加信号的编译模型，在目标窗口（modelsim-object)会显示你选中的编译模型所有的信号，直接在目标窗口选中我们需要观测的信号添加即可，反复操作。

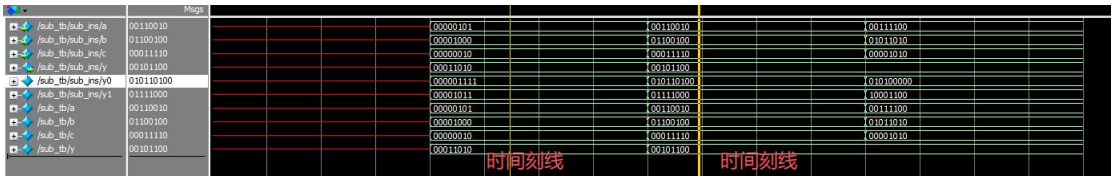


第 07 步: 运行仿真

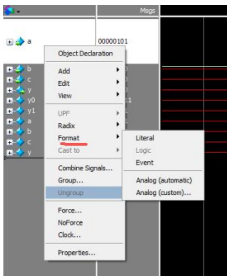
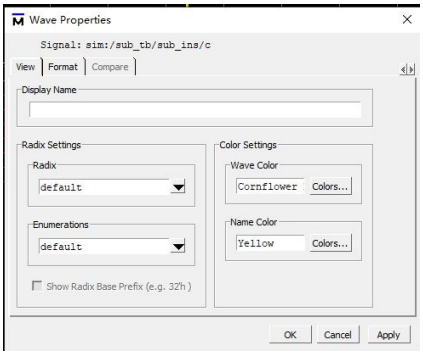
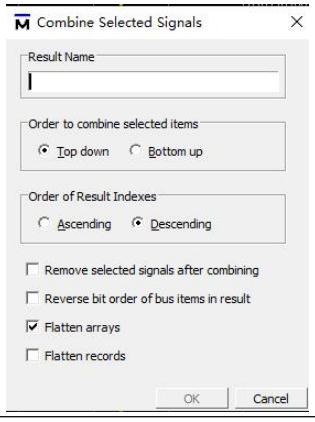
运行操作	描述
重新运行	重新仿真
单步运行	需要设定运行时长，仿真按照我们设定的运行时长进行仿真
连续运行	需要抖动停止运行

第 08 步: 波形分析

在观测波形的时候以当前时刻为基准，所谓当前时刻为我们手动选择的时间基线，通常通过 mark 表示，在实际中当前时刻刻选择多个参考点，通过添加，产出时间刻线选择当前时刻的参考点。



波形分析常用的操作	目的
放大	波形放大
缩小	波形缩小
全屏显示	波形全屏显示
时间测量	添加/删除时间刻线
进制选择	设置信号值的显示格式

信号沿跳变	
信号简单显示	
离散连续信号转换	
波形形态设置	设置波形形态 
信号合并	多个信号可以合并为一个信号 

Note:不管哪种仿真，所有的波形分析一样。

1.2.2 自动化仿真

数字电路设计中一种只能编写的仿真形式叫做自动化仿真，通过脚本命令完成仿真，modelsim 的脚本为 do 脚本。

第 1 步：创建脚本文件

my_sim.do

第 2 步：定位工作路径

```
cd     C:/Users/zxope/Desktop/exam01/prj01/sim
```

第 3 步：定义虚拟库

```
vlib  my_work;
```

第 4 步：虚拟库映射

```
vmap  my_work     my_work;
```

第 5 步：编译所有与仿真有关的文件

```
vlog  -work  my_work    ../src/*.v
```

第 6 步：启动仿真

```
vsim  my_work.sub_tb
```

第 7 步：打开波形显示窗口

```
view   wave  ;
```

第 8 步：删除所有波形

```
delete wave  /*;
```

第 9 步：添加需要的波形

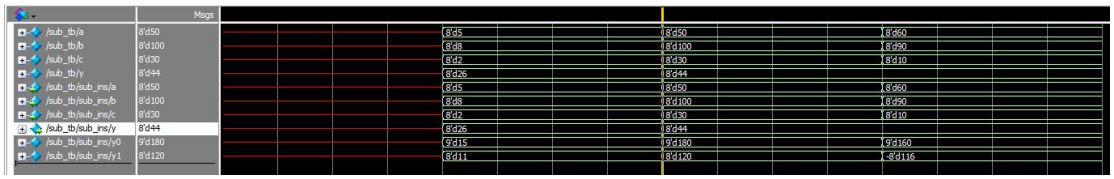
```
add  wave  -dec   /sub_tb/*  ;
add  wave  -dec   /sub/*  ;
```

第 10 步：运行仿真

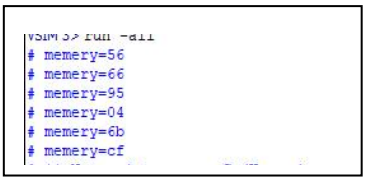
```
run  100ns;
```

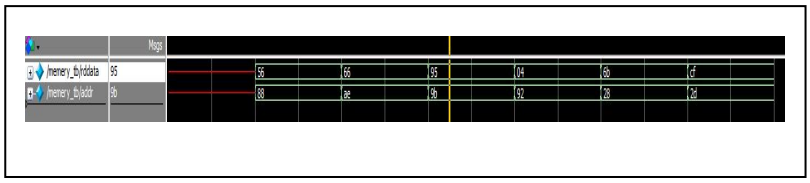
第 11 步：启动仿真器

```
cd   C:\Users\zxope\Desktop\exam01\prj01\sim
vsim  -do   "my_sim.do"
```



仿真案例
标题:存储初始化仿真
要求：
1：reg [7:0] mem [255:0]
2: $display 打印存储器的数据到终端
3：通过 for 循环初始化 mem
4：独立仿真和自动化仿真进行验证





1.2.3 联合仿真

联合仿真是仿真方式之一，无论是新手还是经验丰富的开发者都非常实用，80%开发者都喜欢选择这种方式。

所谓联合仿真就是仿真器结合其他 EDA 工具（通常为集成开发环境）完成仿真操作，通常为集成开发环境+仿真器使用较多。

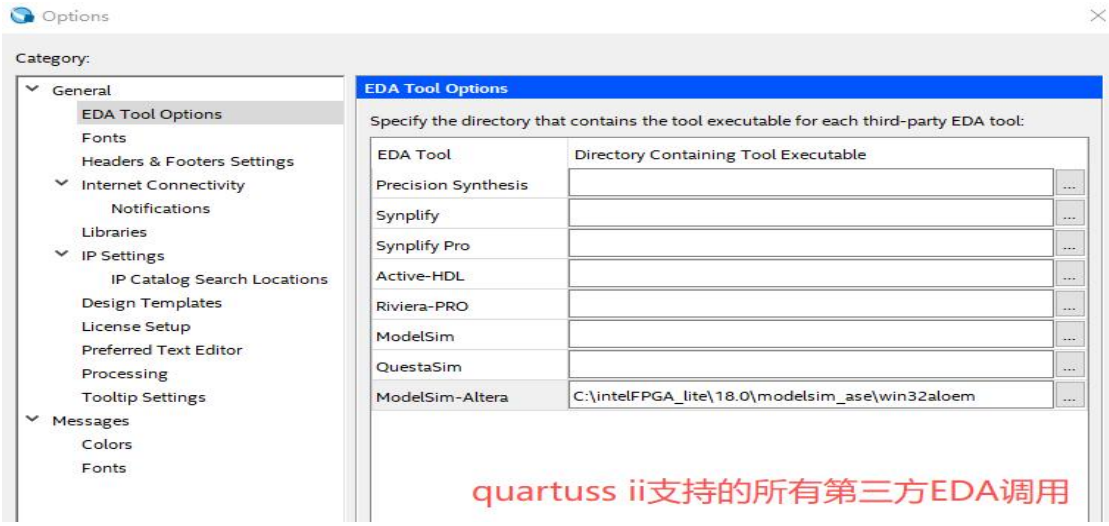
如果是集成开发环境+仿真器的联合仿真，一般为集成开发环境调用仿真器。

联合仿真不需要单独编译我们在仿真中必须使用的额外仿真库，独立仿真和自动化仿真都需要，所以他比较友好。

对于 FPGA 开发者联合仿真基本为集成开发环境+仿真器的形式，我们这里重点介绍这种仿真的操作流程。

第 1 步：完成设计文件和激励文件的编写
第 2 步：在集成开发环境中设置仿真器的安装路径

由于集成开发环境需要调用仿真器，因此必须指定安装路径，便于集成开发环境准确找到仿真器，才能调用。

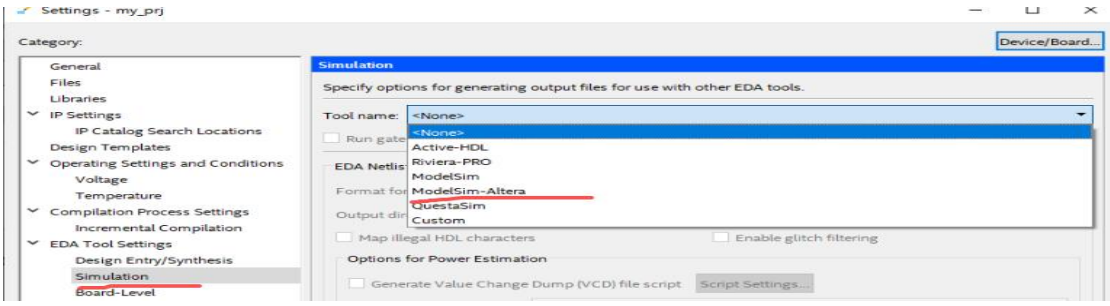


第 3 步：如果有必要需要设置环境变量

有的操作系统在安装 modelsim 的时候没有自动完成环境变量的修改，需要手动修改，如果环境变量不正常是无法正常调用。

第 4 步：设置当前使用的仿真器

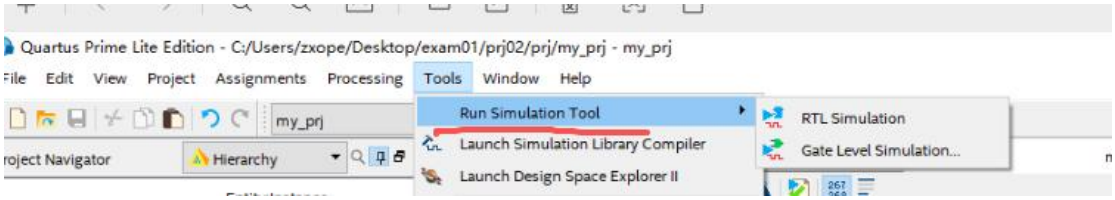
由于仿真器不只有 modelsim 一种，因此我们需要指定使用哪种。



第 5 步：设置 testbench;

任何仿真必须指定才能正常仿真。

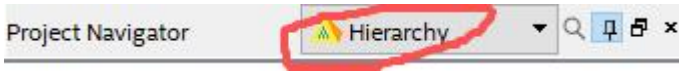
第 6 步：启动仿真

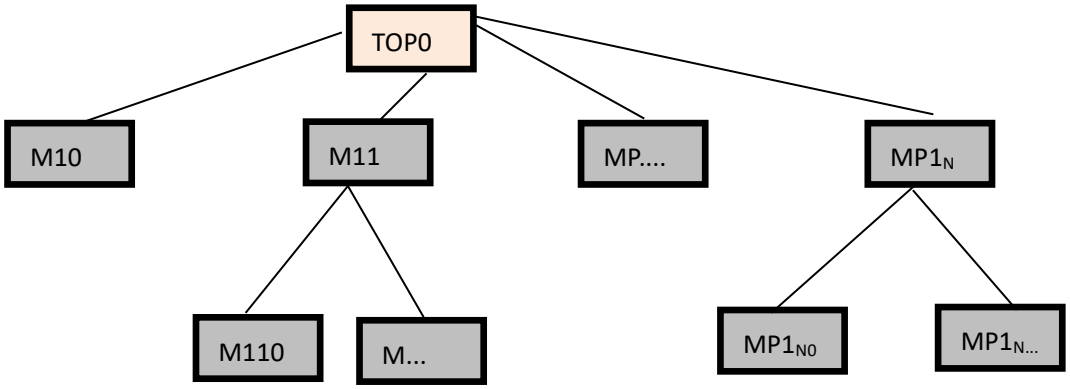


二：HDL 设计基础

HDL 是数子电路设计的一种高效使用的设计方式，任何电路设计以模块为基本单元，所谓的模块就是能够独立完成某一功能的电路，在电路设计中要白痴模块的独立性，从电路系统角度来看，模块本身就是一个电路系统。

在 HDL 设计中，小的电路系统构成大的电路系统，这种特性天然形成一种层次化的结构，因此任何电路具有层次化，在设计中，我们就像搭积木一样，由于电路设计这种属性从而衍生出顶层模块。





电路设计核心要素：

模块核心要素	描述
层次化	电路设计的自然属性
顶层模块	电路设计的自然属性诞衍生出的一种模块，对于一个完整的电路只能有一个顶层模块。顶层模块对于综合器而言为顶层实体。
端口	模块之间通信的桥梁
可配置	模块通用性一种设计方法
建模	HDL 设计中流行的一种概念，通过 HDL 构建电路的模型。

2.1 HDL 建模

在 HDL 设计中广为流程的一种设计思想，是构建任何电路的基本思想，这种设计思想用来描述模块行为运行的特征的设计方法。

如：

```
module   add   #(//模块配置
                parameter   WIDTH   =   8

)
(//模块的端口
                input   [WIDTH-1:0] a,
                input   [WIDTH-1:0] b,
                output   [WIDTH-1:0] c,



);

功能设计（行为特征描述）;

endmodule
```

广为流传的建模方式有 3 种方式，分别数数据流建模，行为建模，结构化建模，任

何电路模块描述功能都离不开这三种方式。

模块功能描述设计方式	描述
数据流建模	简单表示驱动信号，负载信号，变化，条件信号之间的一个逻辑关系，只能用来设计组合电路 每一个 HDL 语言都会提供这样的建模方式。
行为建模	通用的建模方式，相比较数据流建模，这种方式用来描述更复杂的驱动信号，负载信号，变化，条件信号之间的一个逻辑关系，组合电路和时序电路都可以设计
结构化建模	HDL 设计中模块层次化建模的直观体型

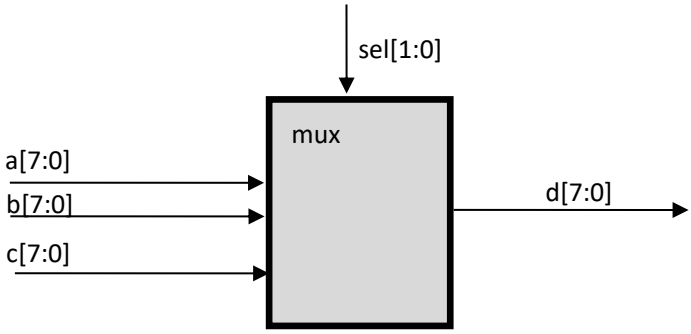
2.1 数据流建模

简单表示驱动信号，负载信号，变化，条件信号之间的一个逻辑关系，只能用来设计组合电路，每一个 HDL 语言都会提供这样的建模方式，并且提供了这样的行为运行特征描述语句。

语言	行为特征运行描述关键字
verilog	**assign**
vhdl	**wen** **with** **直接赋值**

设计案例：3 选 1 多路器
要求：
1：组合电路
2：通过数据流建模
3：真值表

a[7:0]	b[7:0]	c[7:0]	sel[1:0]	d
a	b	c	0	a
a	b	c	1	b
a	b	c	2/3	c




```
module          mux
(
            input   wire    [7:0]   a,
            input   wire    [7:0]   b,
            input   wire    [7:0]   c,
            output wire     [7:0]   d
);
//数据流建模，用来描述 mux3 模块的功能
assign          d       =   (sel==2'd0)         ?           a[7:0]
    :
                    (sel==2'd1)         ?           b[7:0]          :
                                                    c[7:0]          ;


endmodule
```

2.2 行为建模

行为建模是任何 HDL 语言的一种建模方式，提供给我门一种相对于数据流建模更为通用，且复杂的一种建模方式，任何数据流建模设计的电路，都可以用行为建模来设计。

通过这种建模，我们可以描述任何电路行为，简单的，复杂的都可以，既可以设计组合电路又可以设计时序电路。

在 HDL 语言中，通过行为运行特征描述中的进程控制语句，简称进程语句。

语言	行为特征运行描述关键字
verilog	**always** **initial**
vhdl	**process**

2.2.1 组合电路建模

在 verilog 中通过 **always** 进行组合电路设计，具体如下;

```
always        @(敏感信号 0，敏感信号 1,.....)
always        @(*)
always        @*
```

敏感信号列表

条件控制符

运行行为特征描述关键字

设计案例：3 选 1 多路器

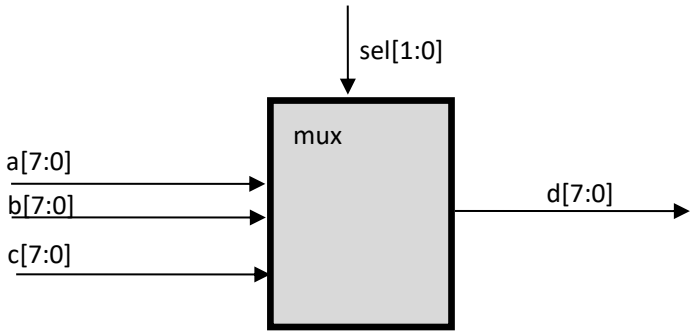
要求：

1：组合电路

2：通过数据流建模

3：真值表

a[7:0]	b[7:0]	c[7:0]	sel[1:0]	d
a	b	c	0	a
a	b	c	1	b
a	b	c	2/3	c

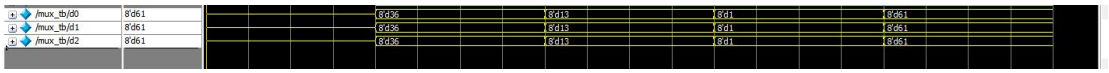



```
module          mux
    (
                    input   wire    [7:0]   a,
                    input   wire    [7:0]   b,
                    input   wire    [7:0]   c,
```

```
                    input wire    [1:0]    sel,
                    output reg     [7:0]    d
    );
    //行为建模，用来描述 mux3 模块的功能
always                    @*
                    if(sle==0)
                         d=a;
                    else if(sel==1)
                         d=b;
                    else
                         d=c;



    endmodule
```



2.2.2 时序电路建模

时序电路就是将组合电路的信号进行采样，并寄存，时序电路设计中有 2 个关键要点：

1：采样

采样方式	描述
沿采样	在采样时钟的边沿采样
电平采样	在采样时钟的稳定的电平阶段采样

Note:在 HDL 设计中时序电路单只边沿采样。

可以看出任何时序电路都必须有采样时钟，因此时序电路建模必须有时钟信号。

2：寄存

当采样到数据后，将其值保存在寄存器，但是第一个采样之前寄存器需要初始化，在时序电路设计中通过复位信号进行初始化，因此时序电路建模有一个关键信号就是复位信号。

综上所述，在任何时序电路建模的过程中涉及到 2 个关键信号，时钟和复位。

在 verilog 中通过 **always** 进行组合电路设计，具体如下：

```
always      @（posedge   clk ）
always      @（negedge   clk ）
always      @（negedge   clk        ）
            if(reset==1)
                ...;
```

```
            else
                ...;
always      @（negedge   clk      ）
            if(reset==0)
                ...;
            else
                ...;

always      @（negedge   clk      or   posedge   reset ）
                if(reset==1)
                ...;
            else
                ...;
always      @（negedge   clk      or   negedge   rst_n ）
                ifrst_n==0)
                ...;
            else
                ...;
```

设计案例：3 选 1 多路器

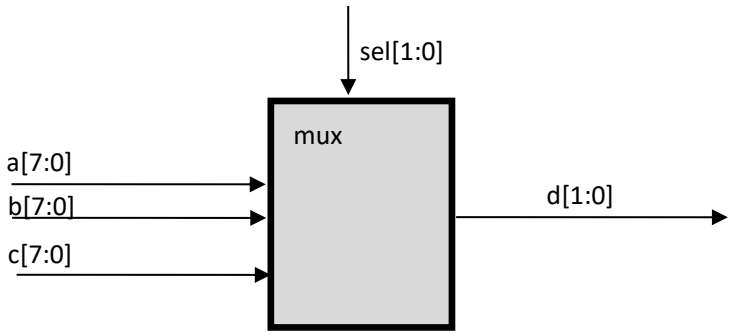
要求：

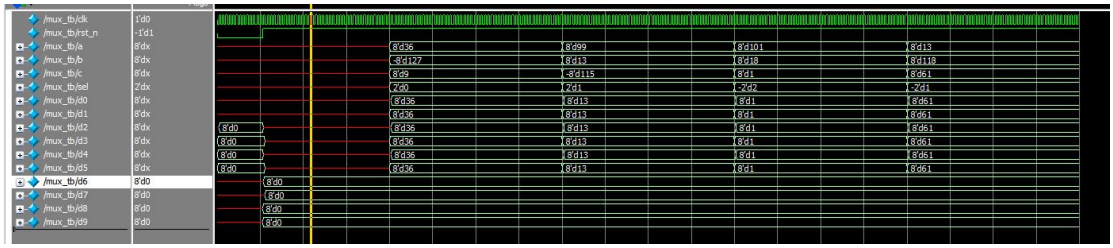
1：时序电路

2：通过行为建模

3：真值表

a[7:0]	b[7:0]	c[7:0]	sel[1:0]	d
a	b	c	0	a
a	b	c	1	b
a	b	c	2/3	c





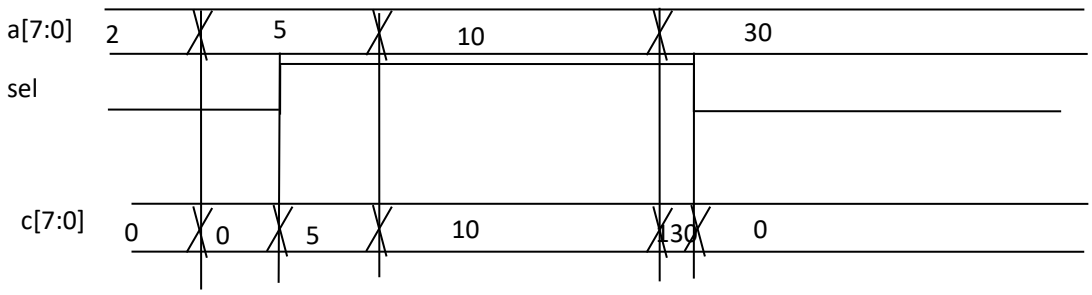
2.2.3 关于组合电路和时序时序特征

在数字电路设计中，组合电路和时序的时序特性存在显著不同，对于我们刚开始接触数字电路设计特别容易混淆的 2 个特征。

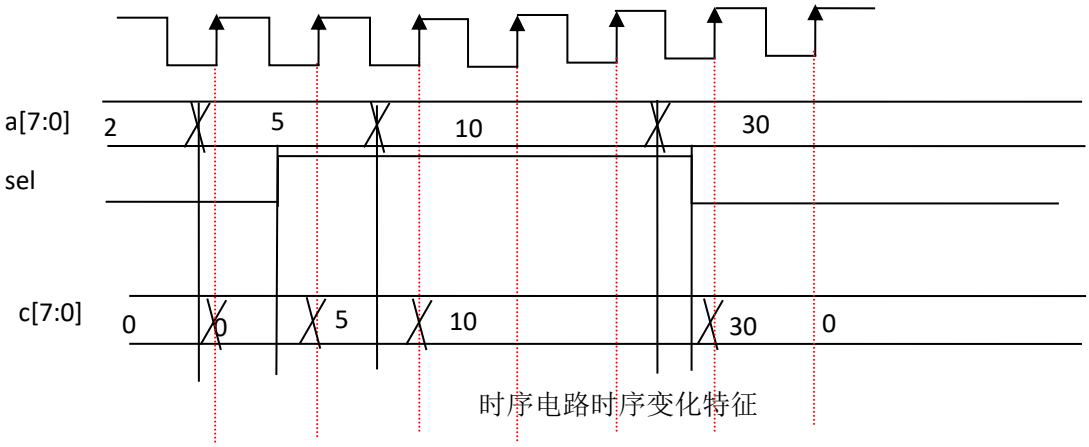
组合电路：理论上，当敏感信号的值发生变化的时候，负载信号的值立即响应。
时序电路：理论上，当敏感信号的值发生变化的时候，负载信号的值不是立即响应，而是在采样时刻发生变化。

```
always    @(*)
          if(sel)
               c= a;
          else
               c=0;
```

```
always    @(posedge          clk)
          if(sel)
               c<= a;
          else
               c<=0;
```



组合电路时序变化特征



时序电路时序变化特征

NOTE：对应仿真器的功能仿真的时序都是模拟理论上的时序。

以上是基本的时序特征，在数字电路设计中我们还需要关注复位信号，复位信号是时序电路设计提供上电时刻寄存器初始化的一种方式。

在电路设计中复位信号需要满足以下要求;

1:复位时钟至少大于电路设计中最慢的时钟的一个时钟周期。
如：电路中最慢的时钟 1khz,则复位时钟必须大于 1ms
2:复位信号的电平变化不能发生在电路设计中任何时钟的时钟沿。
3：复位有异步和同步，在实际这种正确选择复位方式

复位方式	特点	使用场景
异步复位	复位和时钟没关系 真正意义上复位	首选异步复位，但是需要注意复位信号是否满足条件，如果我们不能确定，通常采用同步释放的方法。 有的 IP 不支持异步复位的情况下，必须用同步复位
同步复位	复位必须与时钟同步 本质是寄存器的数据选择	同步复位某种意义上即使在复位信号不理想的情况下，试错率高，但是使用设计资源。 有 IP 的必须使用同步复位

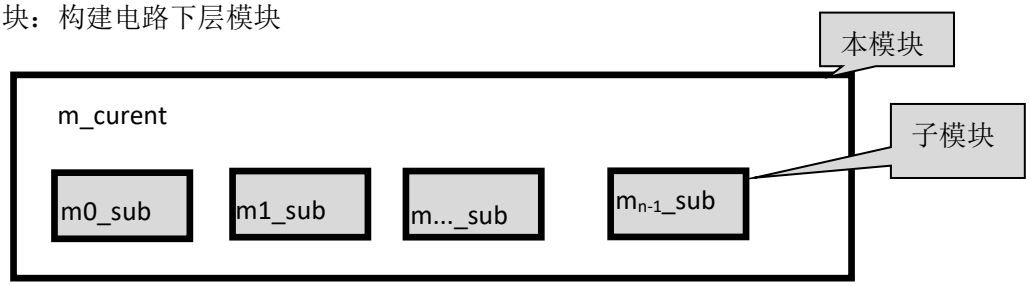
4：复位信号不能有毛刺

2.3 结构化建模

电路模块具备层次化的自然属性，在电路设计中，必须有针对这种属性专用的建模方式，即结构化建模，任何 HDL 语言都必须支持结构化建模。

结构化建模针对的是各个模块之间的层次化的属性，本质就是搭积木，在接口化建模总核心为模块，包括;

- 本模块：构建电路的当前模块（上层模块）
- 子模块：构建电路下层模块



在 HDL 设计中，本模块中的子模块需要以原件的方式存在，换句话说子模块在本模块中我们称之为元件，在本模块中集成子模块对应元件的过程我们通常叫做例化。

元件：本模块中子模块的一种存在形态
例化：原件集成在本模块中的一个过程

在 HDL 设计中，根据结构化建模的建模方式的特点，**标准的例化**需要通过 2 个操作:

- 定义原件

  在结构化建模中，子模块需要先定义为元件。

- 端口映射

  元件作为模块，需要通过端口传递信息，因此元件的端口必须要与本模块的信号进行绑定才能传递信息。

在不同的 HDL 语言总例化操作与不一样，在 verilog 中采用的一种简化的例化方式，vhdl 采用标准的例化方式。

```
模块名     元件名     //元件声明
//端口映射
（
端口映射

）；

如   ：add   add_ins//元件声明
        （


        ）；
```

在 verilog 例化的端口映射有 2 种：
1：顺序映射
2：端口名映射

2.3.1 结构化建模设计案例

标题：异或门，同或门电路 LUT 构建
要求：
1：通过与门，或者，非门，构建异或门
2：与门，或门，非门，通过 Lut 构建
3：通过独立仿真，自动化仿真验证

异或门真值表

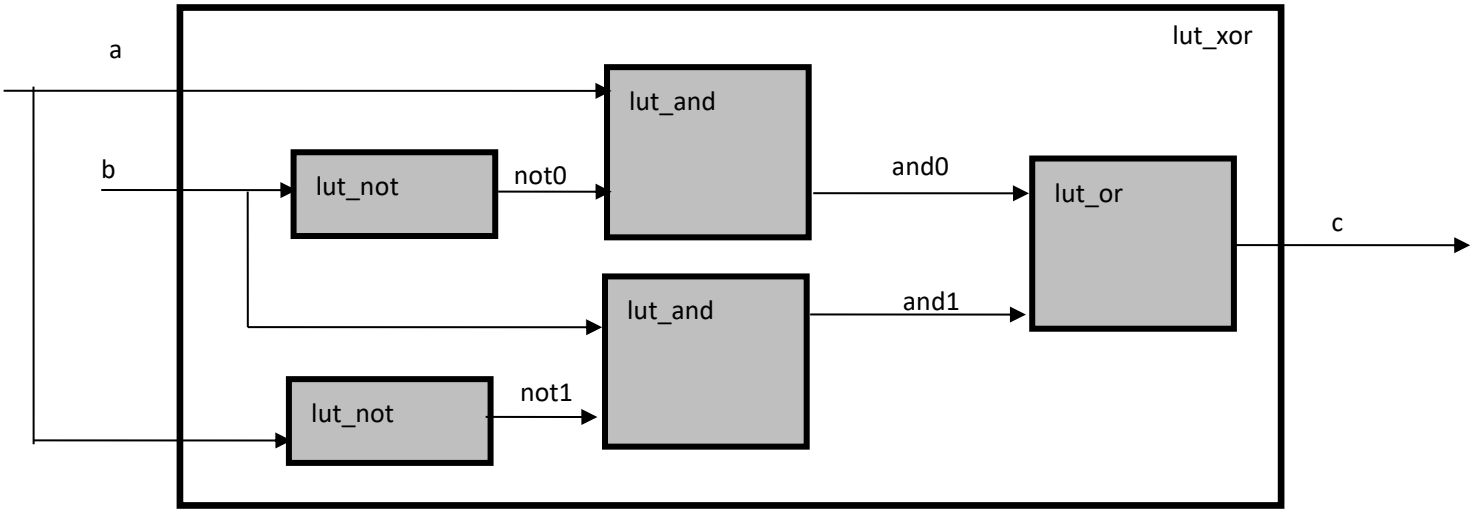
a	b	c
0	0	0
0	1	1
1	0	1
1	1	0

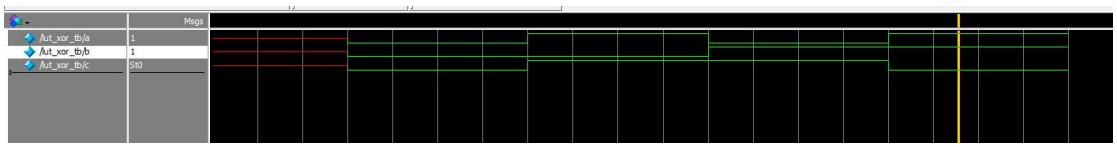
同或门真值表

a	b	c
0	0	1
0	1	0
1	0	0
1	1	1

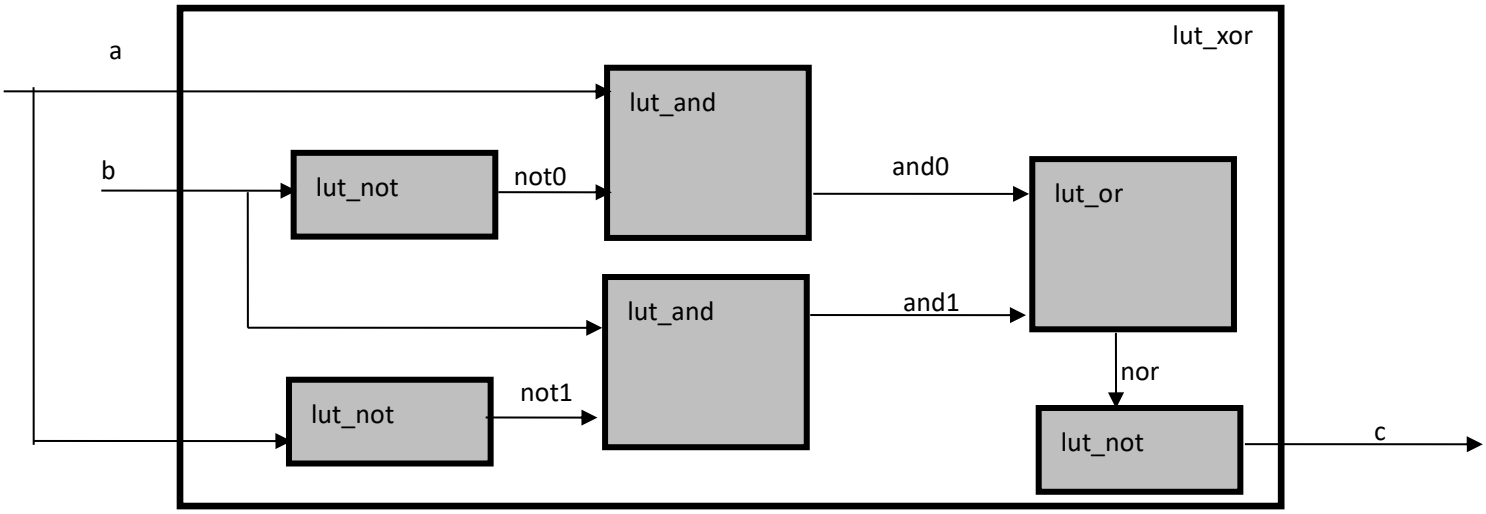
目前公认的异或门的逻辑表示如下：

$c = a\bar{b} + \bar{a}b$

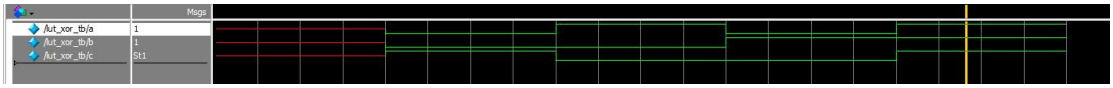




异或门电路 lut 结构化建模



同或门电路 lut 结构化建模

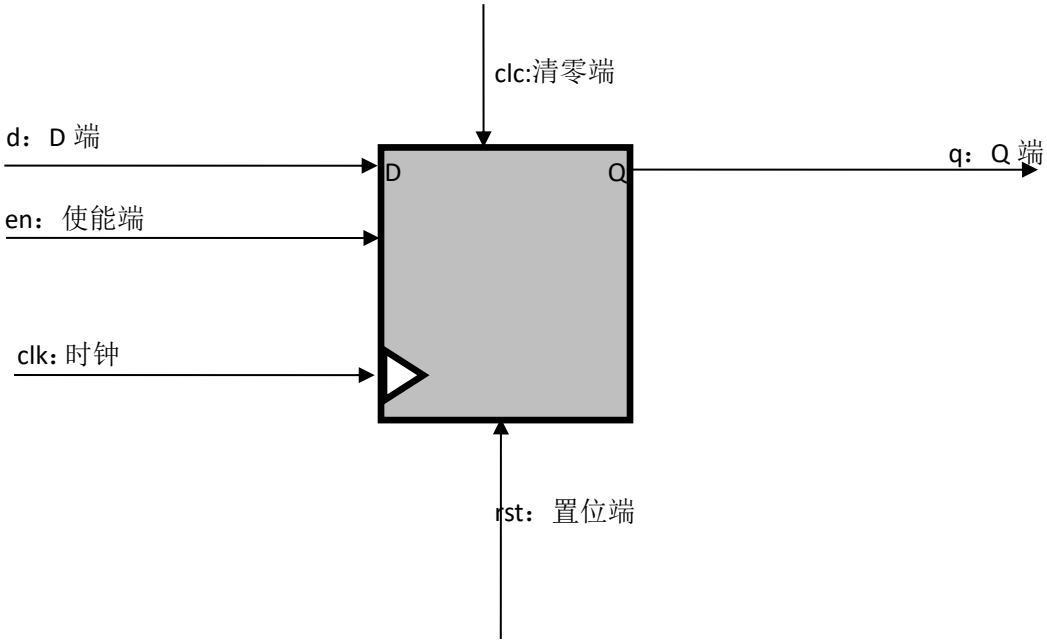


三：基础电路设计

对于数字电路设计，复杂的电路都是由简单的电力构成，在电路设计里相对于设计者而言有几种电路我们为基础电路，除触发器，多路器，译码器，分频器，计数器，对于这几种电路我们必须熟练掌握。

3.1 触发器

在 FPGA 种只用一种触发器，即 D 触发器，我们在 HDL 设计种必须正确使用 D 触发器。




```
always   @(posedge clk    or negedge    clc)
         if(clc)
              q<= 0;
         else if(en)
              q<=d;
```

```
always   @(posedge clk    or posedge    rst)
         if(rst)
              q<= 0;
         else
              q<=d;
```

D 触发器设计案例

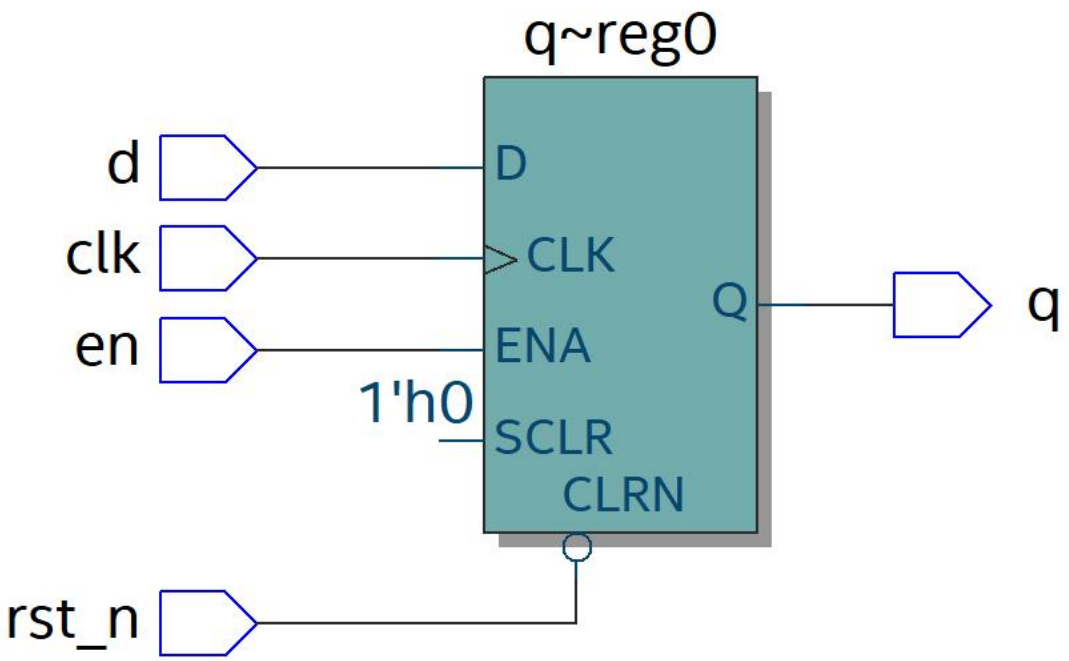
标题：D 触发器设计案例

要求：

1：D 触发器带使能

2：D 触发器置位

3：仿真验证



3.2 多路器

多路器对于数字电路设计者而言位基础，本质是 HLD 语言种的 **if**...**else**..所描述的**控制电路**，最简单的多路器位 mux（单比特 2 选一多路器），在使用多路器的时候我遵循时许设计原则。

1：在 hdl 设计种如果被选择的控制通道较多的化，我们尽量转为并并行译码进行控制

```
if(条件)//通道 0
        ...;
else if(条件)//通道 1
        ...;
else    if(条件)//通道 2
    ...
  else if(通道...)//通道...
    ...;
```

NOTE:在 hDL 设计种，多路器和译码器可以相互转换。

2：多路器优先级的正确设定

在多路器设计种，优先级的设定结合被控制通道的电路的复杂都来设定。

如：4 通道多路器控制电路

```
if(sel[1:0]==2'b00)//第 0 通道
        y=a+b;//第 0 通道被控制的电路
else if(sel[1:0]==2'b01)//第 1 通道
        y=a*b;//第 1 通道被控制的电路
else if(sel[1:0]==2'b10)//第 2 通道
        y=a+b-3;//第 2 通道被控制的电路
else if(sel[1:0]==2'b11)//第 3 通道
        y=a*b+5;//第 3 通道被控制的电路
```

上述电路设计被多路器控制的个**通道控制的复杂度**排名以此是：第 3 通道>第 1 通道>第 2 通道>第 0 通道,正确的优先级设定如下;

```
if(sel[1:0]==2'b00)//第 0 通道
        y=a+b;//第 0 通道被控制的电路
else if(sel[1:0]==2'b01)//第 2 通道
        y=a+b-3   ;//第 2 通道被控制的电路
else if(sel[1:0]==2'b10)//第 1 通道
        y=a*b;//第 1 通道被控制的电路
else if(sel[1:0]==2'b11)//第 3 通道
        y=a*b+5;//第 3 通道被控制的电路
```

3.3 译码器

译码器对于 HDL 设计者而言位基础电路，我们必须熟练正确掌握译码器的设计，它是一种控制电路，在 HDL 种用 **case** 控制语句构建。

在 HDL 设计中译码器的设计，根据所采用的设计方式和设计目标不同，有 3 类译码器。

译码器设计类型	特点	HDL 控制语句	例子
并行	1：时	并　行	case（sel）

译码器	许特性较好
	2：没有优先判决电路
		case	2'b00: out = a; // 互斥分支 1
			2'b01: out = b; // 互斥分支 2
			2'b10: out = c; // 互斥分支 3
			2'b11: out = d; // 互斥分支 4 // 无重叠，综合为 4 选 1 MUX（并行选择）
			endcase

全译码器	1：包含了所有译码的可能
	2：比较稳定
		full case	case (sel)
			2'b00: out = a;
			2'b01: out = b;
			default: out = 4'b0000; // 覆盖 sel=2'b10/11 的情况
			endcase

普通译码器	1:有优先判决电路
	2：时许特性一般
	3：没有要求全译码
		Case	case (sel)
			2'b00: out = a; // 分支 1
			2'b01: out = b; // 分支 2
			2'b00: out = c; // 重叠分支（永远不会执行）
			// 遗漏 sel=2'b10、2'b11 的情况，且无 default
			endcase

在 HDl 语言中并没有针对译码器的设计类型进行定义，针对这种情况，很多综合器支持通过指令指示综合器，我们在使用的时候现用现查。

三八译码器设计案例

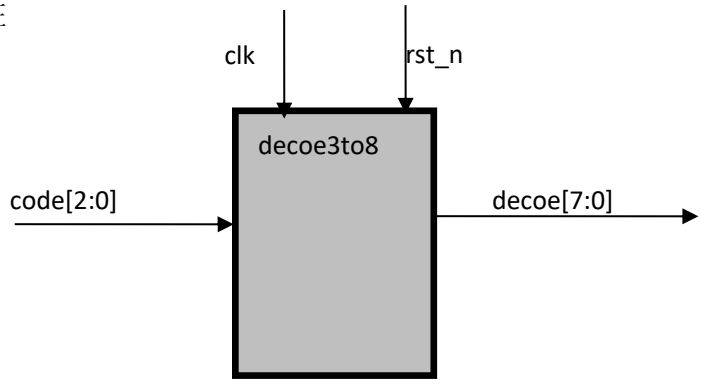
标题：三八译码器设计

要求：

1：全译码

2：用时许电路实现

3：仿真验证



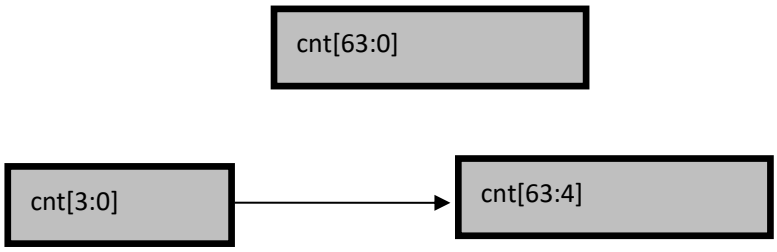
端口说明

端口信号	描述
系统相关	
clk	系统时钟
rst_n	系统复位
译码输入	
code[2:0]	译码输入
译码输出	
decoe[7:0]	译码输出

3.4 计数器

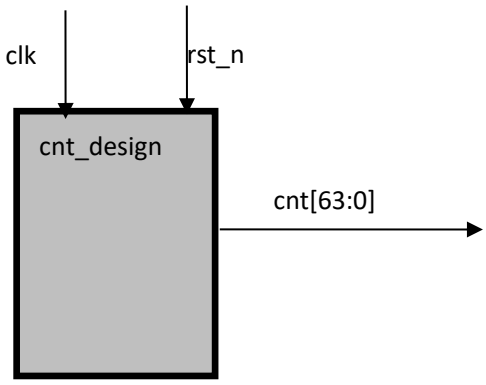
计数器对于 HDL 设计者而言位基础电路，我们必须熟练正确掌握译码器的设计，它是一种控制电路。

预定标技术：对于高位计数器，如 cnt[63:0],这种计数器在复杂电路中应用，为了让设计时许和电路更稳定，通常采用预定标技术。



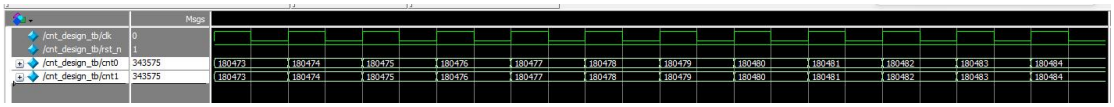
计数器设计案例
标题：计数器设计
要求：
1：64 位计数
2：采用预定标计数

3：仿真验证



端口说明

端口信号	描述
系统相关	
clk	系统时钟
rst_n	系统复位
计数器输出	
cnt[63:0]	计数器输出，默认 64 位

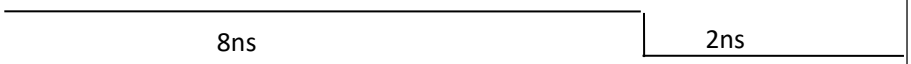
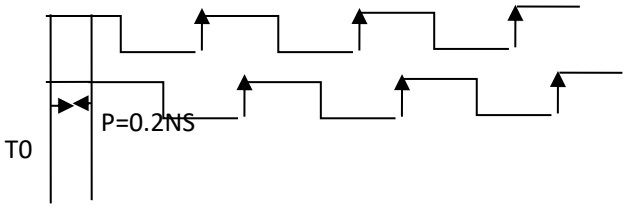


3.5 分频器

时序电路设计时钟信号是必不可少的，时钟信号的好坏直接与时序电路相关联，在实际电路中，涉及到不同的时钟，这就需要分频器电路。

3.5.1 时钟信号与时序电路相关联的属性

时钟关联属性	描述
抖动	普通信号的毛刺对应时钟信号的抖动 定义：某一个时间点较小的时间范围内出现高低电平变化的现象， 抖动时长：我们把这个时间点较小的时间范围叫做抖动时长，它是衡量抖动的唯一指标。如 0.5ps 表示抖动时长为 0.5 皮秒 实际中总抖动时长必须控制较小的范围内。，任何时钟都抖动，0 抖动实际中几乎不存在。
skew	定义：目标时钟偏离源时钟的延迟累计

	在实际电路设计中，需要保证所有寄存器的的 skew **均衡**（skew 值保持一致），在 FPGA 开发中采用一种全局网络时钟树的技术解决此类问题。
占空比	定义：理论上初始化电平在一个时钟周期内所占的比例，通常我们指高电平在一个时钟周期内所占的比例，用百分比表示。 如：占空比 80%，周期 10ns 
相位差	相对于源时钟初始化延迟  绝对相位差：用延迟的时间表示 相对相位差：以某一时钟做参考，相对与该时钟相位变化，通常用 deg 表示 通常我们所说的时钟**相位指相对相位差**
反应速度	电平变化快慢指标，越快时钟信号越好，理想情况下变化时间为 0.但是这个时间的变化不符合实际的物理规律。

3.5.2 分频器电路

所谓分频器就是通过源时钟为参考时钟，生成新的时钟的电路，表示如下;

$$f2(t) = M/N*f1(t-\phi)$$

f1:表示源时钟信号
f2:L 表示生成时钟信号
M/N:分频因子
M：倍频系数
N：分频系数

分频因子 k=M/N	分频系数	频率
<1	M<N	f2<f1

>0		
==1	M==N	f2==f1
>1	M>N	f2>f1

Φ : 相对相位差

分频器在设计中电路设计分 2 种情况：

分频因子 k	实现方法
k>=1	■ PLL ■ DLL
0<K<1	■ PLL:高精度，任意占空比，相位差 ■ DLL:精度要求不高，可选占空比差，相位差要求不高 ■ **用户设计：占空比不可调，0 相位差，精度低**

NOte:用户自定义设计只能实现频率变小，占空比固定，不能变大，且不可以调相位，精度低。

这里我们重点讨论用户自定义分频器设计，公认的分频器分为偶分频器和奇分频器 2 类。

3.5.2.1 偶分频器（K=2,4,6,8,10.....）

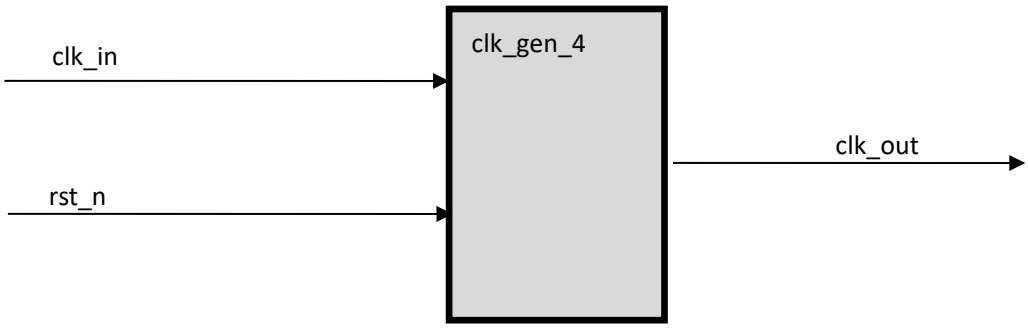
公认的分频电路设计为：

```
always   @(posedge      clk_in      or   negedge      rst_n)
         if(rst_n==1'b0)
              clk_out
         else if(   cnt==K/2 -1     )begin
                                    clk_out<=~clk_out;
                                    cnt<=0;
         end     else
                                    cnt<=cnt+1'b1;
```

设计参数	描述

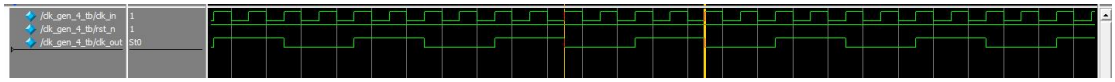
分频因子	0<K<1
占空比	50%
相位	0deg

标题：偶分频器设计
要求：
1：源时钟 50mhz
2: 生成 12.5mHZ 的时钟
3： 占空比 50%，0deg
4:仿真验证



端口说明

端口信号名	描述
系统有关	
clk_in	分频器源时钟，默认 50mhz
rst_n	系统复位，默认低电平有效
分频器输出	
clk_out	分频器输出，默认 12.5mhz



3.5.2.2 奇分频器（K=1,3,5,7,9,11.....）

公认的分频电路设计为：

```
reg clk1,clk2;
assign    clk_out =    clk1|clk2;
```

```
always   @(posedge        clk_in        or   negedge      rst_n)
         if(rst_n==1'b0)
              cnt   <= 0;
         else   if(cnt   ==(K-1) )
              cnt   <= 0;
         else
              cnt<=cnt+1'b1;

always   @(posedge        clk_in        or   negedge      rst_n)
         if(rst_n==1'b0)
              clk1<=0;
         else   if(    (cnt ==(K-1))   |      ( (k-1)/2)     )ckl1<=~clk1;


always   @(negedge        clk_in        or   negedge      rst_n)
         if(rst_n==1'b0)
              clk2<=0;
         else   if(    (cnt ==(K-1))   |   ( (k-1)/2) )ckl2<=~clk2;
```

设计参数	描述
分频因子	0<K<1
占空比	与设计有关
相位	0deg

标题：奇分频器设计

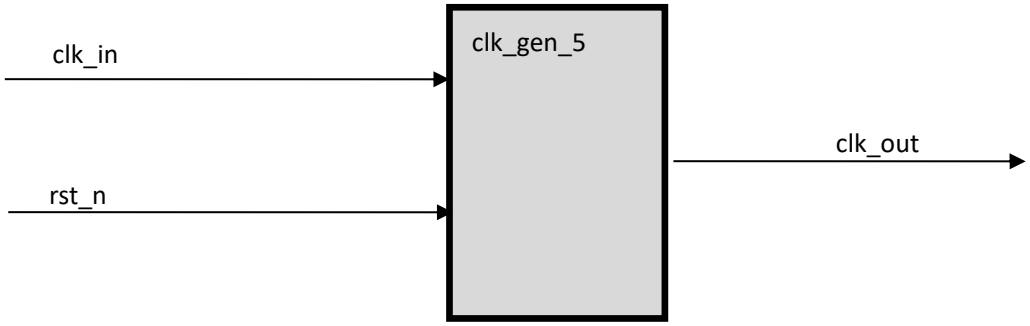
要求：

1：源时钟 50mhz

2: 生成 10mHZ 的时钟

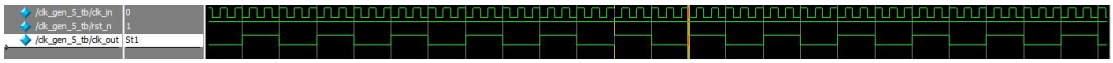
3：0deg

4:仿真验证



端口说明

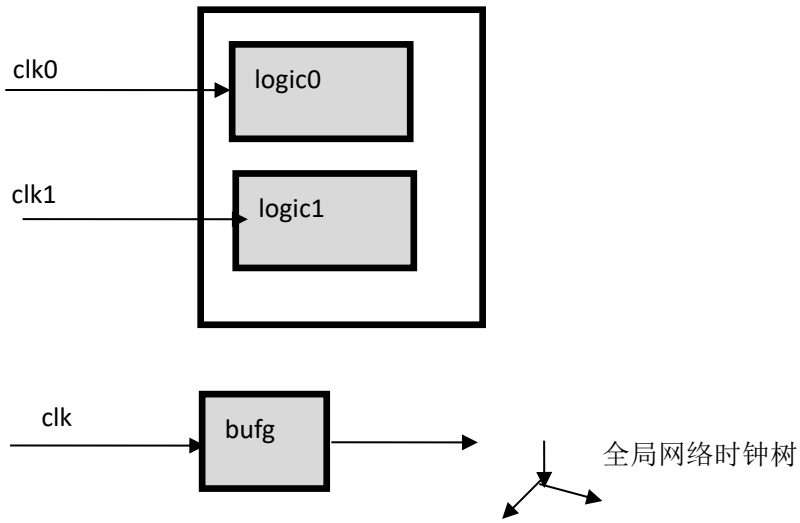
端口信号名	描述
系统有关	
clk_in	分频器源时钟，默认 50mhz
rst_n	系统复位，默认低电平有效
分频器输出	
clk_out	分频器输出，默认 10mhz



3.5.3 时钟分类

在数字电路设计中时钟信号按照应用分为多种时钟，常见的应用如下几种：
1：全局时钟

所谓全局时钟是时钟应用的一个概念版，广义的全局时钟为某一电路由同一源时钟驱动，狭义的全局时钟指的是通过时钟布局，用专用的时钟驱动，比如 FPGA 的全局网络时钟树，通过将时钟信号经过 Bufg 挂接到网络时钟树。



2：组合逻辑输出时钟

所谓的组合逻辑输出时钟，是最常见的一种时钟，理论上来说是可以用于数字电路设计中，但是必须保证**该信号持续稳定**，通常**不建议使用，尽量避免**。

组合逻辑输出时钟分频器设计案例

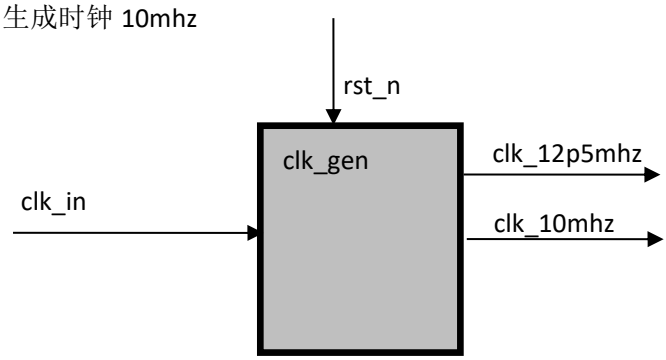
标题：分频器设计

要求：

1：源时钟 50mhz,

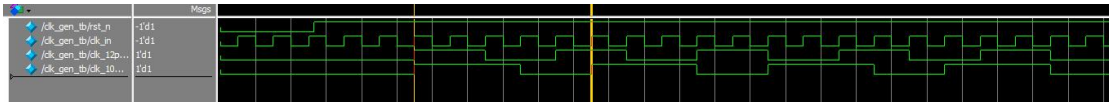
2:生成时钟 12.5mhz

3：生成时钟 10mhz



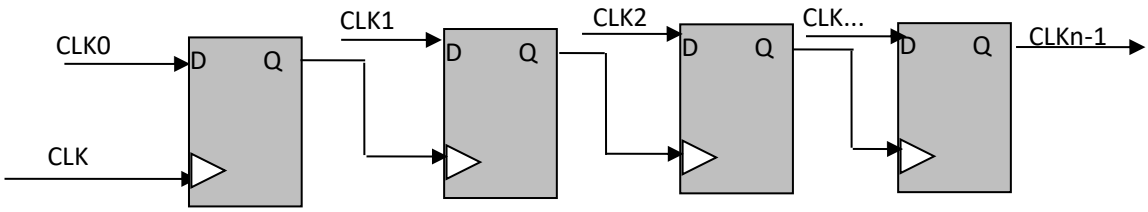
端口说明

端口信号名	描述
系统相关	
rst_n	系统复位，低电平复位
clk_in	源时钟，默认 50mhz
输出生成时钟	
clk_12p5mhz	生成时钟，12.5mhz
clk_10mhz	生成时钟，10mhz



3：行波时钟

行波时钟是数字电路中一种时钟类型，但是**几乎很少使用**，并且适用于**低速时钟信**号。



4：寄存器翻转时钟

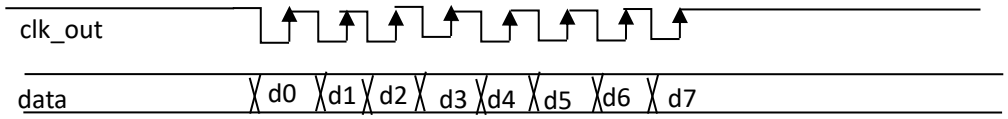
在数字电路设计中，寄存器翻转时钟是一类比较推荐的，稳定的时钟，设计要求不高，

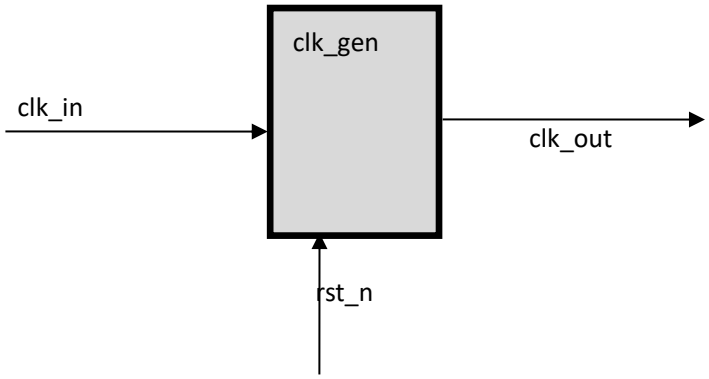
适用于各种电路设计场景，用于定义分频器设计推荐时钟这种时钟生成方法，如偶分频的推荐分频器的设计

5：门控时钟

门控时钟是一类设计时钟，在设计中只能使用一级门控，并且不能有毛刺。

门控时钟设计案例
标题：基于数据发送的时钟设计
要求：
1：数据发送时钟 12.5mhz,默认 100ns 发送一次，每次发送 8 位
2: 源时钟 50mhz
3：时钟设计如下





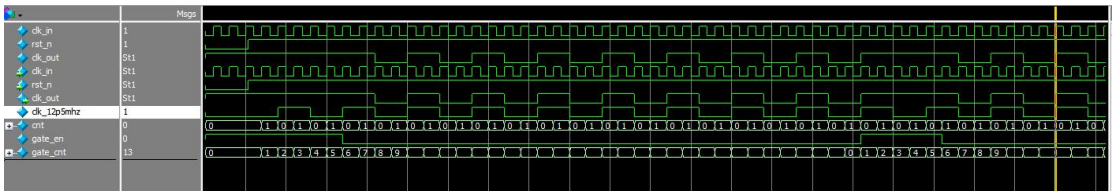
端口说明

端口信号名	描述
系统有关	
clk_in	系统时钟，默认 50mhz
rst_n	系统复位，低电平有效
输出生成时钟	
clk_out	生成时钟，12.5mhz，默认为门控时钟

```
reg   clk_12p5mhz;
        reg    en;
        reg    cnt_gate;
        always     @(posedge          clk_in                or          negedge          rst_n)
                   if(rst_n==1'b0)
                        cnt_gate    <=    0;
                   else    if(cnt_gate==(32+5-1) )
                        cnt_gate    <=    0;
                   else
                        cnt_gate    <=     cnt_gate    +    1'b1;


      assign      en =    cnt_gate >4?    0: 1;

      assing      clk_out    =     en    |    clk_12p5mhz;
```



3.6 加法器

运算电路以加减乘除为基础，乘法器，除法器利用加法器和移位操作实现，有符号数运算也是将减法转换为加法运算，加法器是运算电路中的基础电路。

✧ 1 位半加器

a	b	s(进位)	sum（和）
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

```
sum    =     a ^b;
s      =     a&b;
```

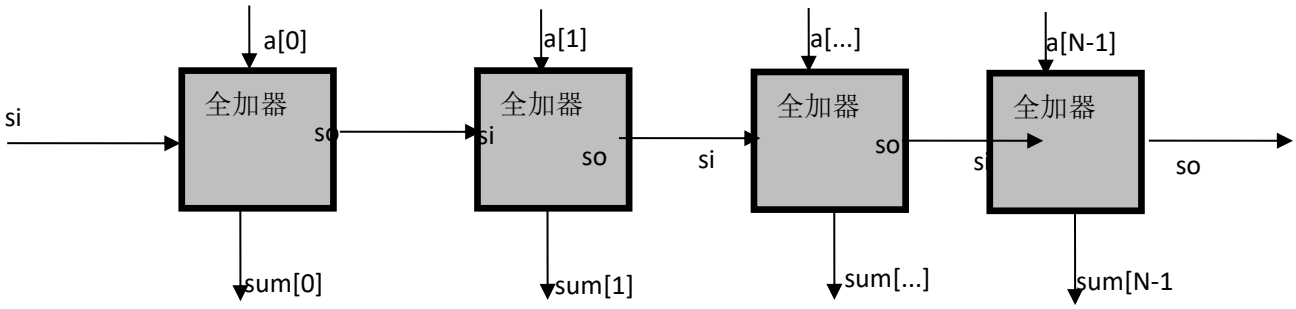
✧ 1 位全加器

a	b	si	s0(进位)	sum（和）
0	0	0	0	0

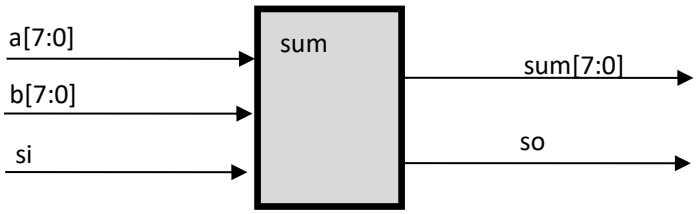
0	1	0	0	1
1	0	0	0	1
1	1	0	1	0
0	0	1	0	1
0	1	1	1	0
1	0	1	1	0
1	1	1	1	1

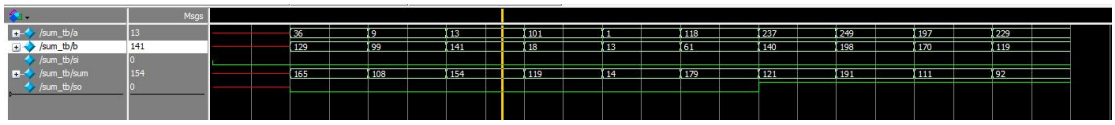
so = s(半加器) + (a*si) + (b*si)
sum=sum(半加器)^si

✧ 多位加法器



加法器设计案例
标题：8 位加法器设计
要求：
1：通过 lut 构建
2:8 位加法器





四：QUARTUSS II 集成开发环境

集成开发环境是提供一个数字电路设计开发的环境，提供一个工具调用与常规操作的**统一管理交互界面**，通常交互界面有可视化管理窗口以及命令窗口，核心功能如下：

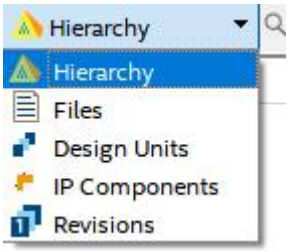
工具调用　　：通过相关工具操作标签可使用该工具（包括软件或者组件）,调用方式可总结为 2 种情况

- 内部调用　　：调用集成开发环境自己附带的工具，比如调用 TimeQuest，Pin Planner.
- 外部调用　　：调用集成开发环境第三方工具,比如：modeslim,Synplify..。

常规操作管理　：作为一个开发环境，为了便于开发，提供一些常规操作管理，比如工程管理，开发流程任务管理，控制塔，IP 管理，

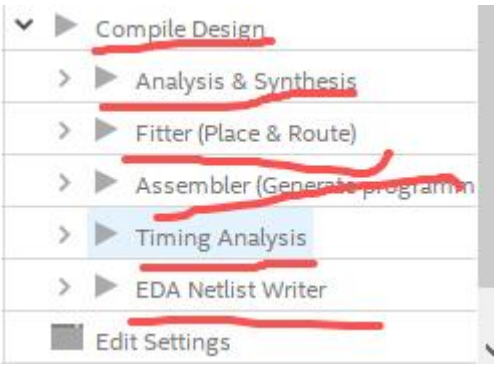
4.1 集成开发环境基本概念与分区

❖ **窗口**：任何软件都是以窗口作为基础，对于窗口而言常见的操作属性为:

操作属性	操作描述
关闭	关闭该窗口
放大	放大该窗口
缩小	缩小该窗口
悬浮	脱离主窗口
标签	窗口内部功能分区 
嵌套	主窗口中可以嵌套其它窗口，有的软件（QUARTUSS II）不支持窗口嵌套，有的软件(vivado)支持窗口嵌套
弹窗	完全脱离任何窗口的单独窗口
隐藏	让对应窗口完全消失在显示区域
显示	让对应窗口完全小时在显示区域

Note:对于任何软件不可能只有一个窗口

❖ **操作标签：**工具调用或者某一操作对应软件界面显示的标签，实际中通过鼠标点击就可以直接调用工具或者执行相关操作。



标签= 标签名 + 图形/符号

Note:操作标签可以分级，比如一级标签，二级标签。

❖ **标题：**软件的名字/版本/文件名

❖ **菜单：**集中管理所有操作标签的一种方式，可以多级菜单

❖ **工具标签：**操作标签的快捷方式

❖ **分区：**任何软件通常的布局是垂直分区。

标题栏：显示标题

Quartus Prime Lite Edition - C:/Users/zxope/Desktop/exam00/prj01/prj/my_prj - my_prj

菜单栏：显示菜单，这里包含类软件所有的操作与工具调用的标签，按照操作的类统一管理操作标签。

File Edit View Project Assignments Processing Tools Window Help

file:文件操作有关的操作与工具调用
edit:文件标记有关操作
view:隐藏或者显示窗口以及窗口标签
project:和工程操作有关，比如往工程当中添加文件，删除文件，压缩工程...
assignments:常见的与分配有关的操作，比如分配器件，分配设置参数，分配引脚...
procsssing: 与开发有关的处理相关操作，比如综合，布局布线，生成下载文件
tools:代用一些高级工具，比如 modelsim 仿真，时序分析....

工具栏：显示常用操作标签的快捷方式，即工具标签

my_prj

其它显示栏：用来灵活显示一些窗口
在 FPGA 开发中，核心窗口有 3 个
工程管理窗口：用来管理工程，QUARTSUS ii 该窗口的操作标签如下：
层次化：显示模块层次结构的
文件：显示工程中所有的设计有关文件

设计实体：显示模块实体
IP：显示 IP 核
版本：显示当前工程的版本
任务栏管理窗口：用来管理 FPGA 开发流程相关的操作任务，QUARTSUS ii 该窗口的操作标签如下：
编译：显示与编译有关的操作任务标签
全设计：显示齐全设计有关的操作任务标签，编译是全设计的一部分
门级仿真：显示与门级仿真有关的操作任务标签
RTL 仿真：显示 RTL 仿真有关的操作任务标签
快速重编译：显示与增量编译有关的操作任务标签

控制台：用户和软件交互的唯一渠道
用户：通过 TCL 脚本告诉 QUARTSUS ii 我们需要干什么
软件：打印输出警告，关键警告，错误，编译过程等相关信号

辅助窗口：IP 管理窗口

4.2QUARTUSS II 软件版本与 FPGA 开发工程属性

◆ QUARTUSS II 版本

QUARTUSS II	特点
web	完全免费，支持的器件库有限
专业版	需要破解，覆盖所有器件库
企业版	需要授权，除了专业版所具备的功能外，还提供了一些高级服务。

◆ 工程属性

工程属性	描述	
工程名	工程的名字	
工程路径	存放工程有关的文件夹	
顶层实体名	工程指定的顶层模块的名字	
器件属性	系列	产品系列
	封装	芯片封装类型
	引脚数	FPGA 上可用的引脚资源
	速度等级	同一型号的芯片差异化分级，比如-1l 表示低功耗等级为 1
	型号	芯片的带好你

4.3 蜂鸣器驱动

蜂鸣器是一种报警设备的发声装置，用来对紧急事件报警，蜂鸣器有 2 类：

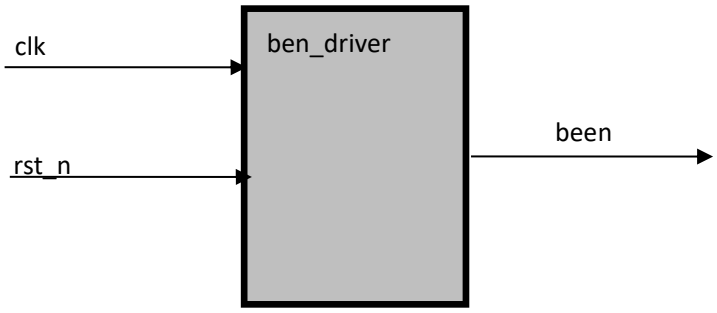
蜂鸣器分类	描述
无源蜂鸣器	内部没有震荡源，蜂鸣器发声需要依赖于交变信号。需要通过方波信号，在交变的时候才能发声
有源蜂鸣器	内部有震荡源，高电平就直接发声

蜂鸣器驱动设计案例

标题：蜂鸣器驱动设计

要求：

1：每 1 秒切换一次频率，频率从低到高在到低

2：频率变化范围

简谱	唱名	标准频率（Hz）	编程常用取整频率（Hz）
1	do	261.63	262
2	re	293.66	294
3	mi	329.63	330
4	fa	349.23	349/350
5	sol	392.00	392
6	la	440.00	440
7	si	493.88	494